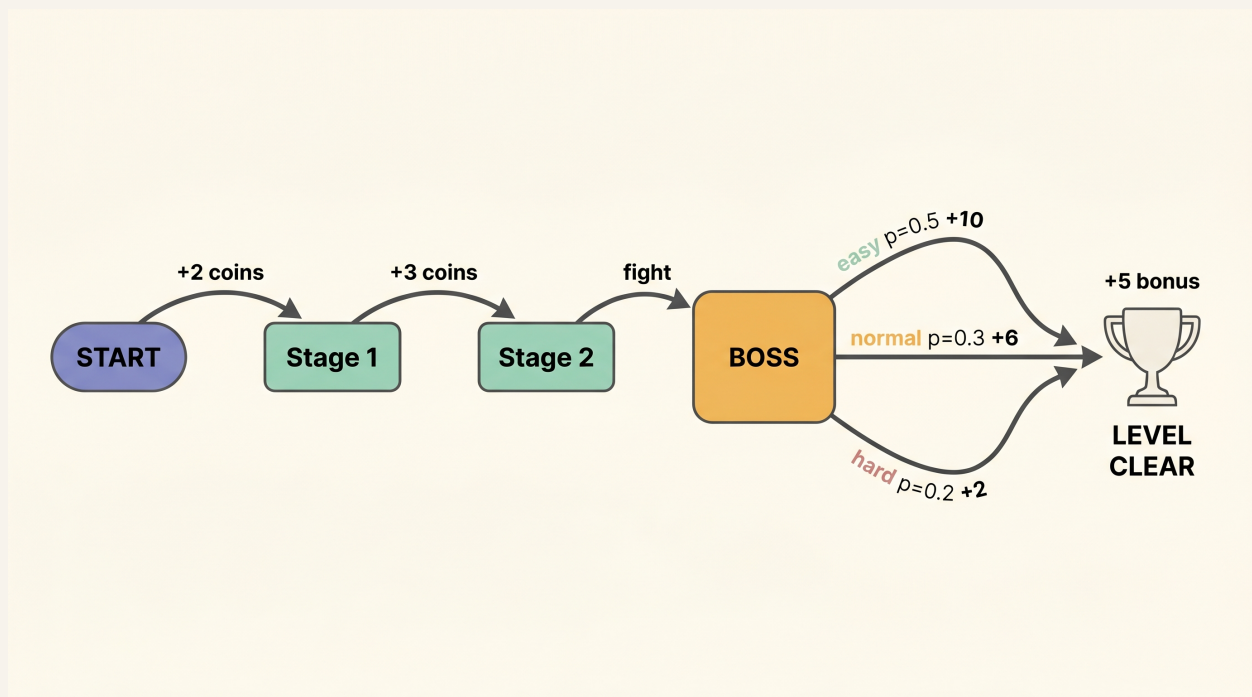


DP, Monte Carlo & Temporal Difference

A second worked example — clearing a level in a mobile game.

You already saw the three methods on a Pune commute. Here is the *same* trio on something more your speed: **beating Level 7 in a mobile game**, where you collect coins and the only gamble is which boss spawns. Different skin, identical idea — and a chance to watch DP, MC, and TD agree on one number from three directions, this time *maximizing* a reward instead of minimizing a time.



The level as a Markov reward process. Fixed coins on every stage – except the **boss**, where the spawn is random (easy / normal / hard). That random branch is the transition kernel P .

The level, as states and rewards

Stage	Coins	Behaviour
START → Stage 1	+2	fixed
Stage 1 → Stage 2	+3	fixed
BOSS	+10 / +6 / +2	easy 0.5 / normal 0.3 / hard 0.2
BOSS → Level Clear	+5	fixed bonus

The question: how many coins should I expect from the *start* of the level? In RL terms, what is $V(\text{START})$? The honest answer must account for the boss being easy most of the time but occasionally brutal.

The true value is fixed by the model:

$$V(\text{START}) = \underbrace{2 + 3}_{\text{stages}} + \underbrace{(0.5 \cdot 10 + 0.3 \cdot 6 + 0.2 \cdot 2)}_{\text{expected boss} = 7.2} + \underbrace{5}_{\text{clear}} = 17.2 \text{ coins.}$$

DP, MC, and TD all chase that 17.2 — watch *how* each gets there.

Dynamic Programming – compute it on paper

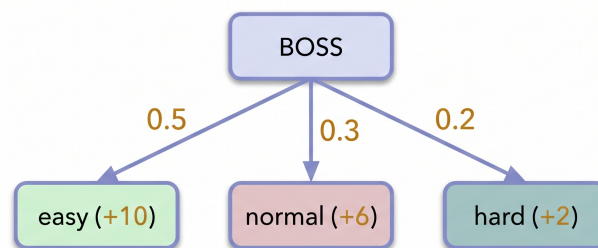
PART 2 • IF YOU KNOW THE SPAWN ODDS

If the game's wiki tells you the spawn rates (easy 0.5, normal 0.3, hard 0.2), you never have to play. Start at the end (0 coins left to earn) and work *backward*, averaging over the boss:

$$V(\text{BOSS}) = \underbrace{0.5(10) + 0.3(6) + 0.2(2)}_{7.2} + 5 = 12.2, \quad V(\text{S2}) = 12.2, \quad V(\text{S1}) = 3 + 12.2 = 15.2,$$

$$V(\text{START}) = 2 + 15.2 = 17.2.$$

Dynamic Programming: average the boss on paper



$$0.5 \times 10 + 0.3 \times 6 + 0.2 \times 2 = 7.2 \text{ coins}$$

$$\text{value of START} = 2 + 3 + 7.2 + 5 = 17.2 \text{ coins}$$

DP needs the spawn odds in advance - then it never has to play.

DP's god's-eye view: average the boss outcomes by their probabilities, then add the fixed coins. No level was ever played.

The catch, again. DP needs the spawn probabilities up front. Real games (and real life) rarely hand them to you — and then you must *learn from playing*. That is where MC and TD come in.

Monte Carlo – play, then average

PART 3 • NO MODEL? JUST PLAY

Don't know the spawn rates? **Play the level many times** and average the coins. Each run totals $2 + 3 + (\text{boss}) + 5$, so easy runs give 20, normal 16, hard 12. Average over many runs:

$$V(\text{START}) \approx \frac{1}{N} \sum_i G_i \rightarrow 0.5(20) + 0.3(16) + 0.2(12) = \mathbf{17.2}.$$

The update, run by run: $V \leftarrow V + \alpha[G - V]$, where G is the *actual* coins that run earned. No probabilities; the game rolled them for you. The cost: you only learn at the end of a run, and a lucky/unlucky boss swings G a lot.

Temporal Difference – update each stage

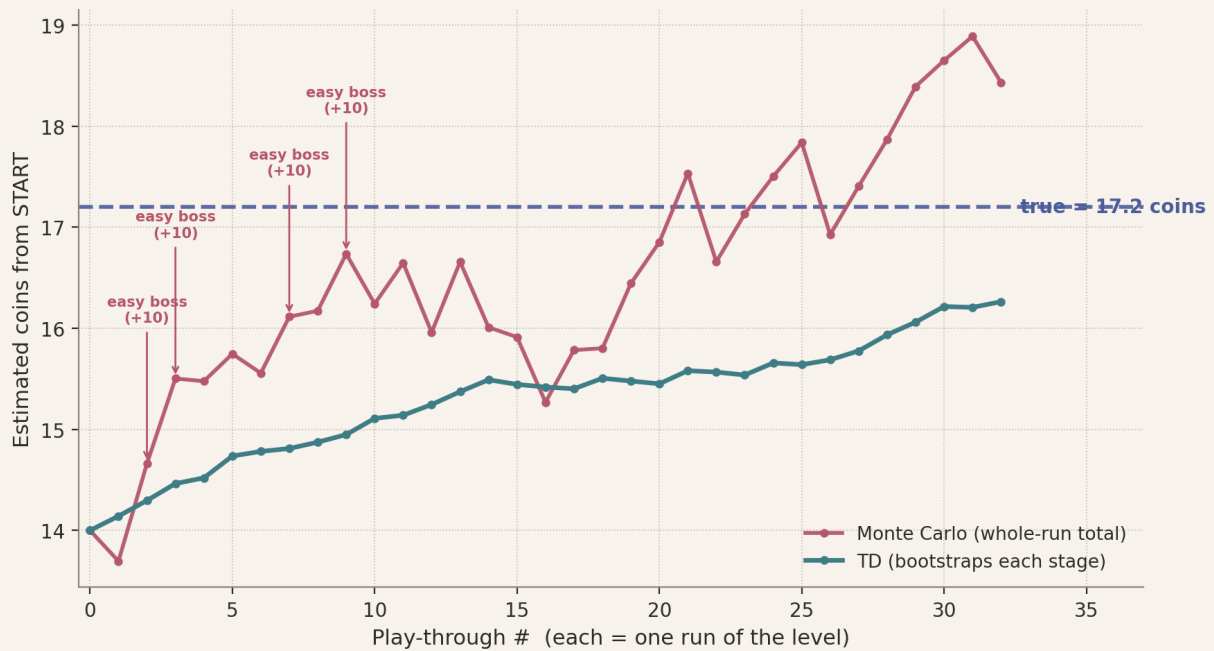
PART 4 • DON'T WAIT FOR THE CLEAR SCREEN

TD refuses to wait for the level-clear screen. After each stage it nudges that stage's value using the coins it just got plus its current guess of what's left:

$$V(s) \leftarrow V(s) + \alpha \left[\underbrace{r + \gamma V(s')}_{\text{one stage + a guess}} - V(s) \right].$$

Hit an easy boss? The stage just before it bumps up *immediately* — no need to finish the run. That is bootstrapping: one estimate corrected by the next.

Same level, two estimators: MC swings with each run, TD settles



Playing the level many times. MC's estimate lurches with every run (the whole-run coin total is noisy, especially the boss); TD leans on steady estimates of the stages ahead and settles faster. Both converge to 17.2.

Same coins, three routes there

PART 5 • THE CONTRAST

	Dynamic Prog.	Monte Carlo	Temporal Diff.
Needs spawn odds P ?	Yes – known up front	No	No
Learns from	all outcomes (the model)	the actual run total	one stage + a guess
When it updates	on paper, offline	at the end of each run	after every stage
Bias / variance	exact (if model known)	unbiased, higher variance	some bias, lower variance

The one line to remember: *all three are estimating the same expected reward $V(s) = \mathbb{E}[G \mid s]$. DP averages it from the model, Monte Carlo averages the runs you actually played, and TD averages one step at a time using its own running guess. Same 17.2 coins — three temperaments.*

RL in Production • Lecture 01 practice • pair with the commute example (Practice #1) and the assignment (Practice #3)